

Taller 7: Interfaces gráficas con JFrame

Sistema de gestión de envíos

1. Integrantes del grupo

- Daniel Ramirez
- Santiago Torres
-

2. Funcionamiento de la interfaz generada

Para este taller construimos un JFrame llamado `VentanaEnvios`, que es el formulario principal del sistema. La idea era que el formulario cambiara según el tipo de envío que el usuario quisiera registrar, en lugar de mostrar todos los campos posibles todo el tiempo.

Para eso usamos un `JComboBox` donde el usuario elige la modalidad de envío: Estándar, Express o Internacional. Cada vez que cambia la opción seleccionada, se dispara un `ActionListener` que revisa cuál modalidad quedó seleccionada y, a partir de ahí, decide qué campos deben aparecer en pantalla.

Ese cambio lo manejamos con el método `setVisible()`, aplicado directamente sobre los campos y sus etiquetas. Por ejemplo, si el usuario selecciona Express, mostramos el campo de Hora Límite y ocultamos Dirección y País llamando a `setVisible(false)` sobre ellos. Si selecciona Internacional, hacemos lo contrario: mostramos el campo de País y ocultamos los otros dos. Con Estándar, el único campo que queda visible es Dirección.

El cambio pasa en tiempo real, sin cerrar la ventana ni recargar nada. Así, la persona que usa el formulario solo ve los campos que necesita llenar para esa modalidad, en vez de encontrarse con una pantalla llena de campos que no le sirven.

3. Cómo está construido (arquitectura)

Para no mezclar la lógica del negocio con la parte visual, separamos el proyecto en paquetes: uno contiene la parte gráfica (la ventana, los componentes de Swing, los listeners) y otro contiene la lógica de los envíos.

Dentro del paquete de lógica creamos una interfaz llamada `Envio`. Esta interfaz obliga a que cualquier clase que la implemente tenga un método `calcularCosto()`, que es el que después usamos para saber cuánto cuesta cada envío según su modalidad.

A partir de esa interfaz creamos tres clases: `EnvioEstandar`, `EnvioExpress` y `EnvioInternacional`. Cada una implementa `Envio` y define su propia versión de `calcularCosto()`, con la fórmula que le corresponde a esa modalidad. El envío internacional, por ejemplo, puede tener un costo base distinto al del envío estándar, y cada clase se encarga de calcular eso por su cuenta.

Por último, hicimos una clase `GestorEnvios` que administra todos los envíos que se van registrando. Por dentro usa un `ArrayList<Envio>` para guardarlos. Como el `ArrayList` está tipado con la interfaz y no con una clase específica, puede guardar objetos de cualquiera de las tres clases sin ningún problema.

4. Cómo se consume el servicio

Todo el flujo arranca cuando el usuario llena el formulario y le da clic al botón Registrar. En el método asociado a ese botón, lo primero que hacemos es capturar el texto que el usuario escribió en los campos que estén visibles en ese momento, usando `getText()` sobre cada `TextField`.

Después revisamos qué modalidad quedó seleccionada en el `JComboBox`. Con esa información, un condicional decide qué clase instanciar: si es Estándar creamos un `EnvioEstandar`, si es Express un `EnvioExpress`, y si es Internacional un `EnvioInternacional`. Como las tres clases comparten la interfaz `Envio`, en el resto del código las tratamos todas como si fueran del mismo tipo, sin importar cuál se haya creado realmente. Ahí es donde entra el polimorfismo: el programa no necesita saber en cada punto qué clase exacta tiene entre manos, le basta con saber que cumple con la interfaz `Envio`.

Una vez creado el objeto, lo mandamos al `GestorEnvios` para que lo agregue a su `ArrayList`. Inmediatamente después llamamos al método `actualizarTabla()`, que recorre todo el `ArrayList` de envíos registrados y, por cada uno, llama a su `calcularCosto()` y pinta la fila correspondiente en el `JTable`: modalidad, datos ingresados por el usuario y el costo final calculado.

Así, la tabla siempre queda sincronizada con lo que hay guardado en el `GestorEnvios`, sin tener que actualizarla campo por campo de forma manual.